for page_num, page in enumerate(doc, start=1): try: text = page.get_text() if text.strip(): results.append({"type": "text", "content": text}) else: # Image-only page pix = page.get_pixmap() img = Image.frombytes("RGB", [pix.width, pix.height], pix.samples) text = pytesseract.image_to_string(img) if text.strip(): results.append({"type": "text", "content": text}) for img_index, img in enumerate(page.get_images(full=True)): xref = img[0] base_image = doc.extract_image(xref) image_bytes = base_image["image"] ext = base_image.get("ext", "png") # default fallback media_type = f"image/{ext.lower()}" print(f"[PDF IMAGE] Page {page_num}, image {img_index+1}: {media_type}") image_b64 = base64.b64encode(image_bytes).decode("utf-8") image_res = call_llm(image_b64) results.append({ "type": "image", "page": page_num, "media_type": media_type, "content": image_res }) except Exception as e: print(f"[WARN] Page {page_num} failed: {e}") return results this will maint order or not

ChatGPT said:

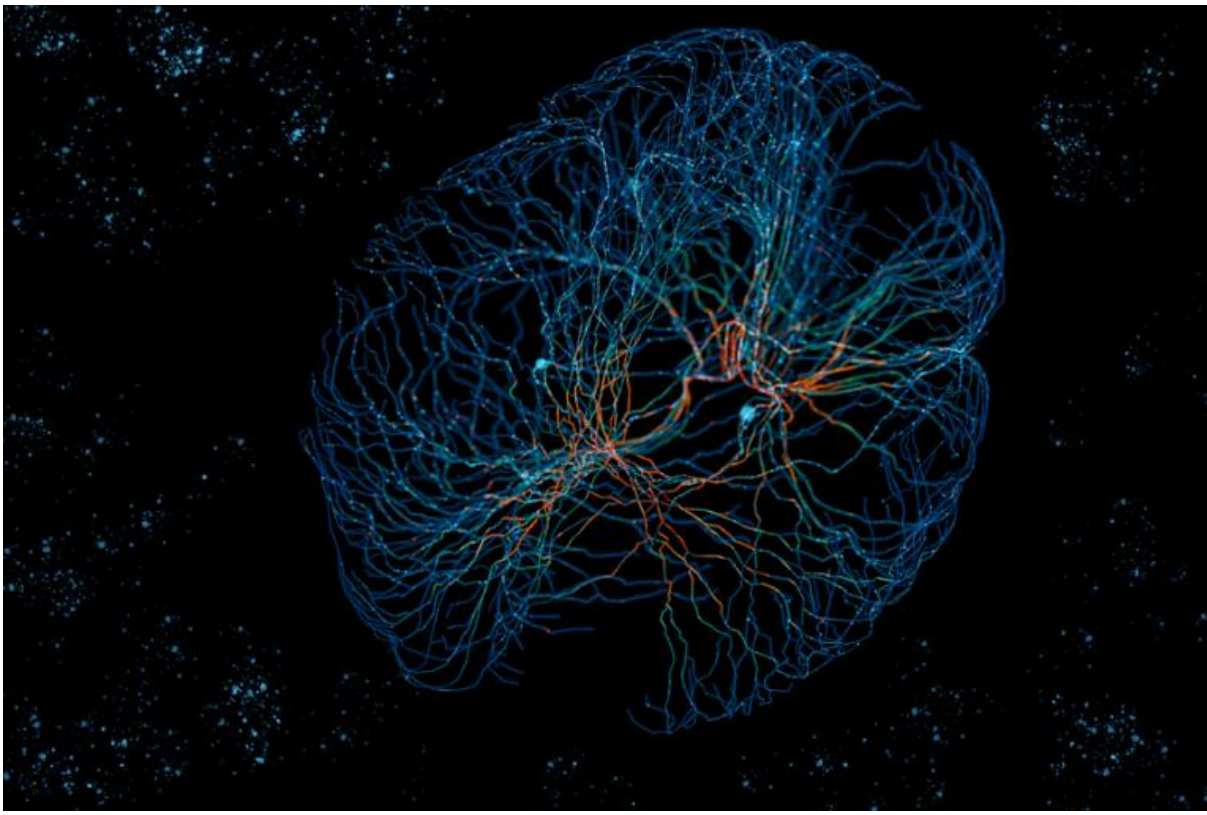
Yes — your current load_pdf function **mostly maintains the order of content**, but there are some subtleties:

Let's break it down carefully:

How it processes pages:

1. for page_num, page in enumerate(doc, start=1): → iterates pages in order (page 1 → last page).
2. Within each page:
    - It first tries text = page.get_text().

- If there is text, it appends



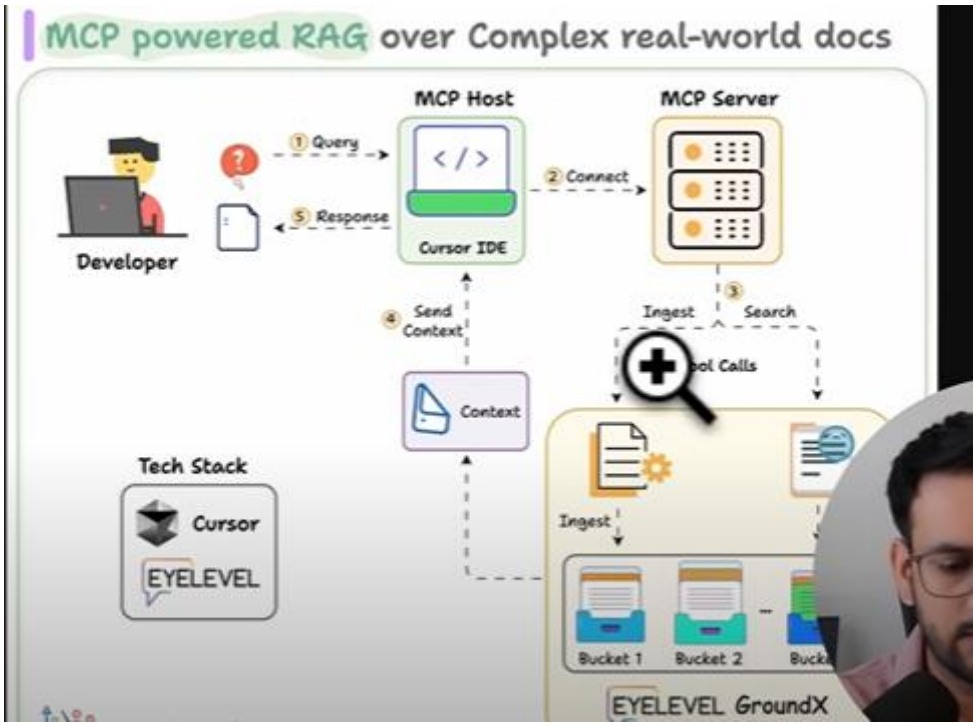
  a **text block** to results.

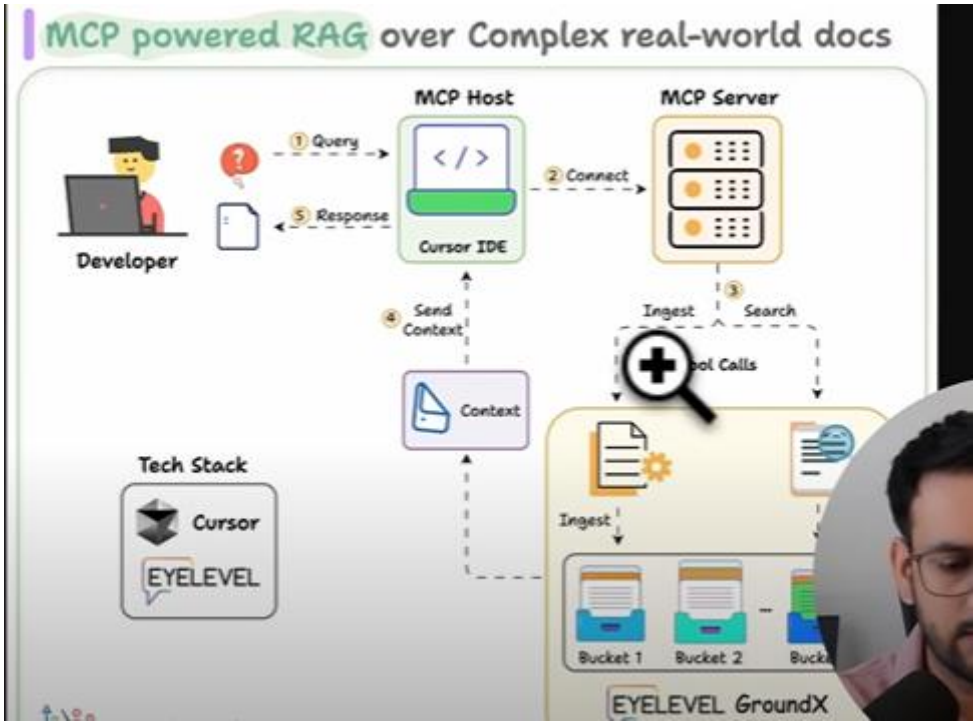
- Else, it treats the page as image-only:
  - Runs pytesseract OCR on the page image → appends text if any.
  - Loops over page.get_images(full=True) → extracts images and appends them **after the OCR text**.

Implications for order:

- **Text-first**: If a page has both text and images, your current code **only appends text from get_text()**. Images will only be appended if get_text() is empty.
- **Image-only page**: OCR text comes first, then images.

- **Page order is preserved** overall (page 1 → page 2 → … → last page).





Potential issues:

1. **Mixed content pages**: If a page has both text and images, images will be ignored entirely unless text is empty.

2. **Multiple images per page**: Images are appended in the order returned by page.get_images(full=True) (usually top-left → bottom-right, but PyMuPDF doesn't guarantee exact visual order).
3. **OCR text and embedded images order**: OCR text is always **before** images on the same page, which might not match the visual layout.

✅ **Recommendation to strictly maintain visual order:**

If you want **true visual order** (text interleaved with images as they appear on the page):

- Use page.get_text("dict") or "rawdict" → gives **blocks with type=0 (text) or type=1 (image)**.
- Iterate blocks in order → append to results.

Example snippet for order-preserving extraction:

```
page_dict = page.get_text("rawdict")

for block in page_dict["blocks"]:

    if block["type"] == 0:  # text

        text_content = " ".join([" ".join([span['text'] for span in line['spans']]) for line in block['lines']])

        if text_content.strip():

            results.append({"type": "text", "content": text_content, "page": page_num})

    elif block["type"] == 1:  # image

        xref = block["image"]

        base_image = doc.extract_image(xref)

        image_bytes = base_image["image"]

        ext = base_image.get("ext", "png")

        media_type = f"image/{ext.lower()}"

        image_b64 = base64.b64encode(image_bytes).decode("utf-8")

        image_res = call_llm(image_b64)

        results.append({

            "type": "image",

            "page": page_num,
```

```
        "media_type": media_type,
        "content": image_res
    })
```

This **fully preserves the order** as seen on the page.

If you want, I can rewrite your **full load_pdf function** to handle **text + images in exact visual order**, including OCR fallback only when needed. This will make it production-ready.

Do you want me to do that?